

Exercise 1

Write a Java console program that:

Asks the user to enter their weight (kg) and height (meters) via the keyboard.

Calculates the BMI using the formula: $BMI = \text{weight} / (\text{height} * \text{height})$

Prints the BMI value rounded to 2 decimal places.

Prints the corresponding BMI category to the user.

Exercise 2

Create a class Book (in the same file as Main) containing fields: title (String), author (String), price (double).

In the Main class, prompt the user to enter the number of books they want to input. Then, use a loop to collect the information for each book from user input.

As soon as the input for a book is completed, immediately display that book's information before proceeding to the next one.

Exercise 3

Create a class Product with fields: id, name, description, price.

- Create 2 constructors: one no-argument constructor and one with all fields.
- Create getters and setters for the Product class.
- Create a method doSale() in the Product class: when this method runs, the program asks the user to enter a discount amount for the product, then updates the product's price by subtracting the discount from the old price.
- In the file containing the main method, create two Product objects, product1 and product2, using the two constructors above (product1 with the no-argument constructor, product2 with the full-argument constructor).
- In main, set values for product1 using the getter/setter methods.
- Run doSale() for both objects.
- Print the information of both objects after the price update.
- Compare the prices of the two objects.

Exercise 4

Write a Java program to support the sales department's employee data entry process. The program should collect Employee ID, Name, Position, Basic Salary, and Sales Done for each salesperson. Based on the employee's sales figures, the program calculates a commission according to the following rules:

- Sales $\geq 10,000$ – 30% of basic salary
- Sales $\geq 8,000$ – 30% of basic salary

- Sales $\geq 6,000$ – 20% of basic salary
- Sales $\geq 4,000$ – 10% of basic salary

Based on the calculated commission, the program computes the total salary by adding the commission to the basic salary. Display the employee's details as follows:

Employee ID:
Employee Name:
Designation:
Basic Salary:
Sales Done:
Total Salary:

Exercise 5

Design a small area calculator. Create a program to calculate the area of shapes such as circles and squares.

- All classes may be placed together in a single `.java` file (no package is required).
- The shape and the value of the radius or side length must be entered by the user.
- If the user does not provide any value, an appropriate message must be displayed.
- If the user provides the required value(s), calculate the area of the specified shape.
- Display the result as follows:
 - User Number:
 - Shape:
 - Value of radius/length:
 - Calculated Area:
- The User Number must increase with every user who runs the program.
- Package the files into a `.jar` file (either via the command line or using an IDE).
- Run the `.jar` file from the command line.

Exercise 6

Create several small classes to practice fields, constructors, and methods:

1. **Person class:** Create a class `Person` with fields `name`, `age`, and `gender`. Define a method `getInfo()` to print the information of a `Person` object. Create and use several different `Person` objects and call `getInfo()` to print their information.
2. **Student class:** Create a class `Student` with fields `name`, `age`, `studentId`, and `grade`. The class must support 4 constructors: no-argument; `studentId` and `name`; `studentId`, `name`, and `age`; and all four fields. Write a method `calculateGPA()` that calculates the average grade based on a set of scores (an array may be used).
3. **Simple Bank:** Create a class `BankAccount` representing a bank account with a field `balance`. Add `deposit()` and `withdraw()` methods to change the account balance.

4. **Computer class:** Create a class `Computer` with fields `brand`, `model`, and `price`. Define a method `displayInfo()` to display the computer's information.
5. **Circle class:** Create a class `Circle` with a field `radius`. Write methods `calculateArea()` and `calculatePerimeter()` for the circle.
6. **Animal class:** Create a class `Animal` with fields `name` and `sound`. Define a method `makeSound()` to print the sound the animal makes.
7. **Product class:** Create a class `Product` with fields `name`, `price` (`Integer`), and `quantity` (`Float`). Add a method `calculateTotalPrice()` based on price and quantity.
8. **Car class:** Create a class `Car` with fields `brand`, `model`, `year`, and `price`. Write a method `calculateDepreciation()` to calculate the car's depreciation over time, using the rule: each year of age reduces the price by 5%.
9. **Employee class:** Create a class `Employee` with fields `name`, `employeeId`, `position`, and `salary`. Write a method `calculateAnnualSalary()` to calculate the annual salary based on the monthly salary field.
10. **Main menu:** Create a `Main.java` file (with all the classes above placed in the same file) containing a `main()` method to run the application. In the application, display the following menu:

Please choose a feature:

1. Create a `Person` object and display its information
2. Create 4 `Student` objects and display each student's average grade (using 4 different constructors)
3. Create a bank account and deposit/withdraw
4. Create a computer object and display its information
5. Create a circle and display its perimeter and area
6. Create an animal object and print its sound
7. Create a car object and print its depreciation
8. Create an employee object and print their annual salary

Each feature should prompt the user for input, and the object should be created based on the user's input data.

11. **(Advanced) List Management:** Create a class `PersonList` to manage a list of `Person` objects. Add methods to add, remove, and search for a `Person` in the list.

If this bonus exercise is completed, add it to the menu above as feature 9.

Exercise 7

Lifemaxi is an insurance company. It employs insurance agents who provide customers with information about various policies and loan amounts. As a developer, you are tasked with building a Java application that helps a loan agent calculate whether a customer is eligible for a loan, and how much.

Design a Customer class and the eligibility-checking logic below, and build a program that takes a customer's age, gender, and profession (Self-Employed/Professional/Retired) as input and calculates the loan amount they are eligible for, per the following table:

Age category	Gender	Profession	Personal assets	Loan amount eligible
16–25	M/F	Self-Employed / Professional	> 25,000	10,000 (15,000 if Professional)
26–40	M	Self-Employed / Professional	> 40,000	25,000 (30,000 if Female)
41–60	M/F	Self-Employed / Professional	> 50,000	40,000
Over 60	M/F	Self-Employed / Retired	> 25,000	Self-Employed: 35,000 – Age × 100; Retired: 25,000 – Age × 100

If the customer's personal assets do not meet the minimum threshold for their age category, they are not eligible for a loan.

Exercise 8 — Point Class for a Drawing Application

Assume you have been asked to develop a drawing application. This application should be able to draw various shapes such as circles, squares, parallelograms, rectangles, and more. Each shape is a collection of many points. So, you decide to create a `Point` class in the application. The features to implement in the `Point` class are as follows:

1. Since a point is formed by two coordinates on a plane, include `x` and `y` as fields in the `Point` class.
2. Implement methods such as `setX()`, `setY()`, and `displayPoints()` in the `Point` class to represent the behavior of the `Point` class.
3. Implement the necessary constructors to initialize `Point` objects when they are created.
4. Create two `Point` objects and compare their `x` and `y` coordinates. If the coordinates are the same, display "The points are the same"; otherwise, display "The points are different".

Exercise 9 — Text Processing Class

To process text, build a class `TextDocument` with a single field: a character string (its text content).

Requirement 1: Build the constructors `TextDocument()` and `TextDocument(String text)`.

Requirement 2: Build a method that counts the number of words in the text.

Requirement 3: Build a method that counts the number of occurrences of the letter 'A' (case-insensitive) in the text.

Requirement 4: Normalize the text according to the following standard: there is no leading or trailing whitespace, and there are no 2 or more consecutive whitespace characters in the middle of the text.